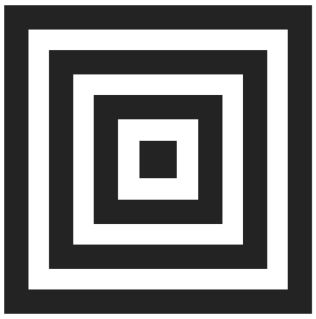


THE NTH LAYER

The AI-Native Software Development Lifecycle

Rebuilding Software Delivery Around the Constraint That Moved

Code ships faster. Customers are not necessarily seeing value sooner. Deciding what to build, testing it, releasing it and measuring the result still run at human speed.

Matthew Dewstowe · October 2026

1. Executive Summary

You are paying for AI tools and engineering time whose extra output is not yet reaching customers or revenue. Your engineers can now produce code faster than your organisation can specify, review and release it. The Software Development Lifecycle (SDLC) was built to ration the expensive part, writing code. AI has made that cheap, so the constraint has moved to deciding what to build, verifying, releasing and measuring it. The evidence on delivery points mostly one way: with high AI adoption, developers merge 98% more pull requests, but review time rises 91% and company-level delivery shows no significant improvement (Faros AI, 2025, vendor telemetry) [8].

Writing and testing code is only about 25% to 35% of the time from idea to product launch (Bain, 2025) [3]. AI "amplifies what's already there", and delivery stability has not yet improved with it (DORA) [1]. Why now: while 40% of software lifecycle decision makers use AI in code generation, only 6% use it in deployment decisions (Futurum, 2026, self-reported survey) [15].

While that gap stays open, AI spend buys output that waits. Our thesis, which can be proved wrong: adopt AI agents and change nothing else, and output rises while lead time stays flat or worsens. AI-native, in Nth Layer's definition, means rebuilding every stage of the SDLC [4].

Three changes do most of the work:

- Every request becomes a hypothesis with a number attached, judged afterwards as won, lost, inconclusive or stopped early.
- The six stages run as a loop. Each leaves a short written record that the next stage reads, building an audit trail as the work is done [2].
- Controls are enforced by infrastructure, not memory, and accountability stays with the named person who approves a release.

We test this on one product area over 12 weeks with three gates. Hypotheses and a baseline come first, then plans, checks and release gates, then live work against targets. If flow measures have not improved by week 8, you stop with no further commitment. Flow measures are credible by week 12; revenue and retention need a quarter or more.

Three questions for your leadership team:

1. Since the team adopted AI coding tools, how much sooner do customers get new value?
2. Where does finished work wait longest before customers see it?
3. What did last quarter's product spend return in revenue, retention or cost?

Sharp answers mean you can see where the constraint is. Vague ones mean you cannot yet see where it has moved.

The business outcome, in board terms: new value reaching customers sooner; the same R&D budget testing more ideas and closing losers early; every release tied to revenue, retention or cost; and a clear go or no-go decision at week 8.

What this means for you. CEO: a bounded test with a decision at week 8. CPO: product judgement at the centre of delivery. CTO: AI investment visible in flow, with enforced controls.

2. The Constraint Moved

Anthropic's playbook says the traditional SDLC "was designed to maximize efficiency in an era where the most time-consuming and expensive stage was writing and implementing code, which is no longer the case" [2]. Bain's 2025 report reached the same conclusion from the other direction: writing and testing code "only accounts for about 25% to 35% of the time from initial idea to product launch", and so far "the savings have been unremarkable" [3].

The SDLC runs from idea to a feature in customers' hands: plan, design, build, test, deploy and maintain. Speed up the build alone and work piles up in review and release.

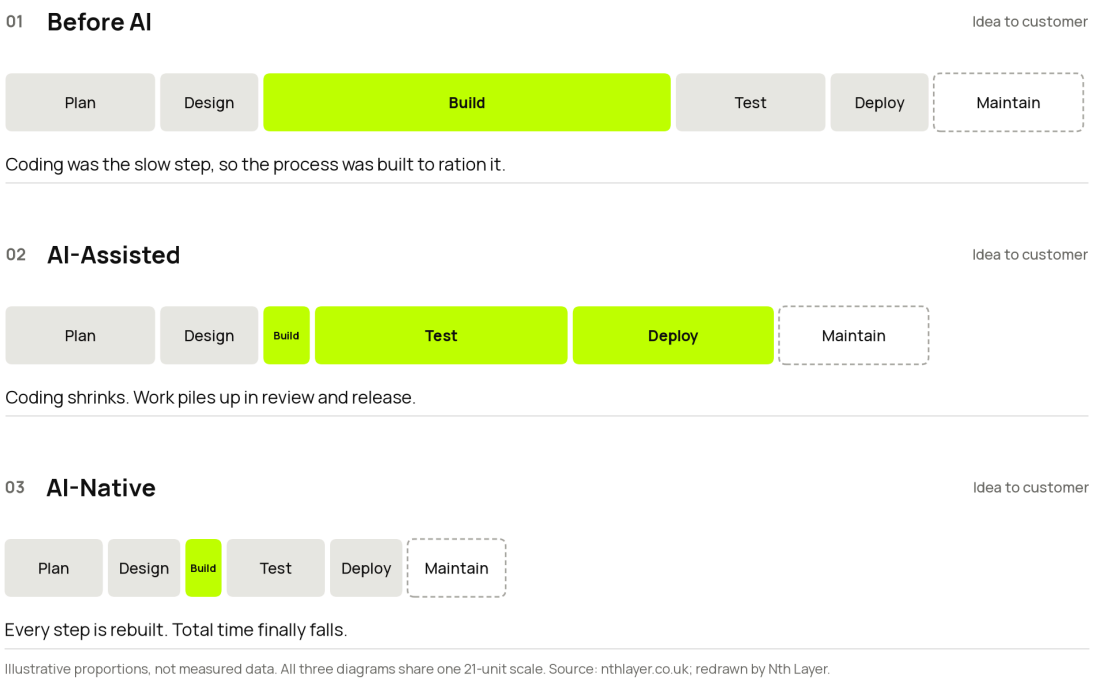


Figure 1. Idea to customer before AI, AI-assisted and AI-native. Illustrative: stage widths are unsourced Nth Layer assumptions. Bain puts writing and testing code at about 25% to 35% of idea-to-launch time [3].

Source: Nth Layer's own model, see [4].

Two things follow. Deciding what to build becomes the most valuable step, because building the wrong thing is now fast too. In agentic coding, "People decide what to build, and the agent decides how to build it" [17]. And line-by-line review stops working: reviewing each line by hand "can't keep up once agents write most of the diff" [2].

We state the thesis in a form that can be wrong. Organisations that adopt AI agents and change nothing else will show rising output, flat or worse lead time, and no way of saying what the extra output bought. If their lead time falls materially, for example by more than 30% with no rise in defects reaching customers, we are wrong.

3. The Evidence

Controlled studies disagree, and self-reported speed is unreliable.

Coding Got Faster

In a 2023 lab task, developers with GitHub Copilot "completed the task 55.8% faster than the control group" [5]. At Anthropic, merged pull requests per engineer per day rose 67% after Claude Code was adopted across engineering (2025) [6], and by October 2026 "code commits per engineer have risen the fastest, coming in at 5x their velocity as six months ago" [7]. Both are Anthropic's internal data, and it adds its own caveat: "More code does not necessarily mean better code or business impact" [7].

Delivery Has Not Kept Pace

Faros AI's 2025 analysis covered telemetry from more than 10,000 developers across 1,255 teams. "Developers on teams with high AI adoption complete 21% more tasks and merge 98% more pull requests, but PR review time increases 91%", and Faros found no significant correlation between AI adoption and improvement at the company level, while bugs per developer rose 9% with AI adoption [8]. This is vendor telemetry and correlational.

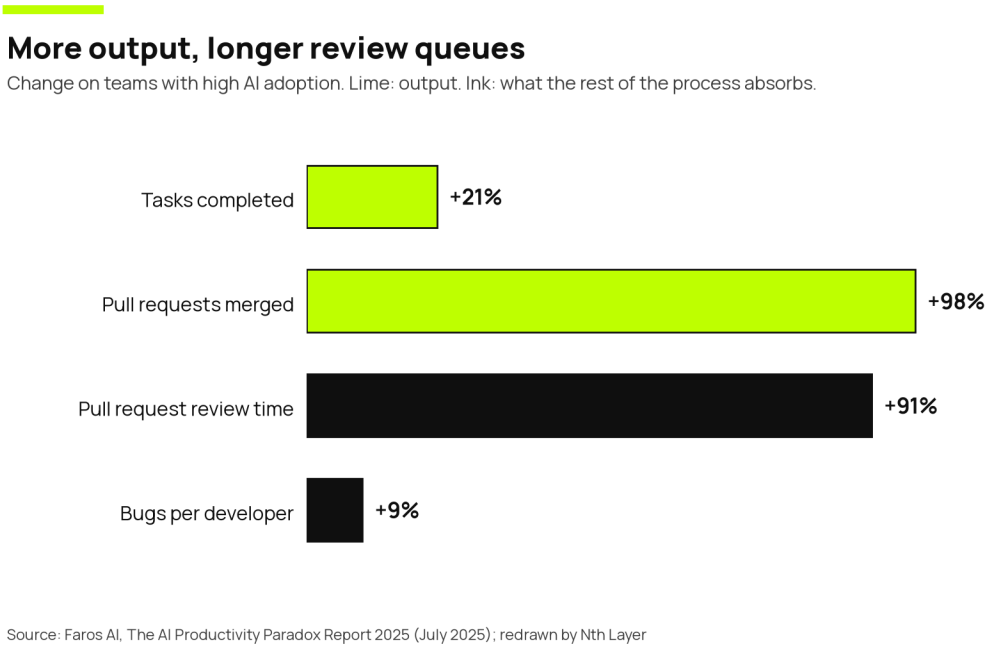


Figure 2. On teams with high AI adoption, pull requests merged rose 98% and review time rose 91% (Faros AI, 2025).

Source: Faros AI, The AI Productivity Paradox Report 2025 [8]; redrawn by Nth Layer.

Atlassian's 2025 survey of 3,500 developers and managers, which is self-reported, found developers "saving 10 hours a week using AI and losing 10 hours a week to inefficiencies" [9]. Google's DORA research estimated in 2024 a 1.5% reduction in delivery throughput and a 7.2% reduction in delivery stability "for every 25%

increase in AI adoption" [10]. By 2025 throughput had turned positive, but stability had not. DORA's 2025 summary is the one we build on: "AI doesn't fix a team; it amplifies what's already there" [1].

Feeling Faster Is Not Being Faster

In METR's 2025 randomised trial, experienced open-source developers took 19% longer to complete real tasks with AI tools, while believing afterwards that AI had made them 20% faster [11]. METR's 2026 follow-up estimates an 18% speed-up for returning developers (confidence interval crossing zero), but calls its new data an unreliable signal [12].

Quality and Governance Lag

Stack Overflow's 2025 survey found that "More developers actively distrust the accuracy of AI tools (46%) than trust it (33%)" [13]. Veracode's 2025 vendor research found that in 45% of tasks the model introduced a known security flaw [14]. In Futurum's self-reported 2026 survey of 839 software lifecycle decision makers, 40% use AI in code generation but only 6% use it in deployment decisions [15]. Futurum also reports (2026, self-reported) that "75% of organizations have already experienced a production incident in which AI-generated code, AI agents, or AI tooling was a contributing factor", while only 18% have standardised or mastered agent governance [16].

4. Requests Become Hypotheses

Today nothing states in advance what a request is for, so nothing can be judged afterwards. A request cannot fail; a hypothesis can. Whoever spots the problem states it as a prediction with a number attached:

We believe [this change] will produce [this outcome] for [these customers]. We will know by [this measure, from this starting point] within [this time]. If we are wrong, we will [do this].

Outcome	What it means	What happens
Won	The measure moved as predicted.	Keep it, and record why it worked.
Lost	The measure did not move, or moved the wrong way.	Close it. The cheapest thing this model produces.
Inconclusive	The measure was too noisy, or no baseline was captured.	Fix the measurement, not the feature.
Stopped early	Enough was learned mid-flight to know it will not pay.	Close it before it is built.

The product owner owns the hypothesis and its resolution; the engineer owns the plan. A hypothesis closed before delivery counts as a success, not a slip, provided the reason is recorded; closures are reviewed each quarter so the rule cannot be gamed. Expect the roadmap to shrink before it speeds up.

Low-volume B2B products rarely give clean verdicts. Samples are small, so treat results as directional, use leading indicators such as usage of the new capability, and record inconclusive rather than force a verdict.

5. A Loop, Not a Chain

The playbook puts the shift in one line: "Instead of a linear flow, the process becomes a loop, and AI is embedded at each point" [2]. The six stages stay, but each now ends with a short written artefact that the next stage, person or agent, reads. Together they form the audit trail.

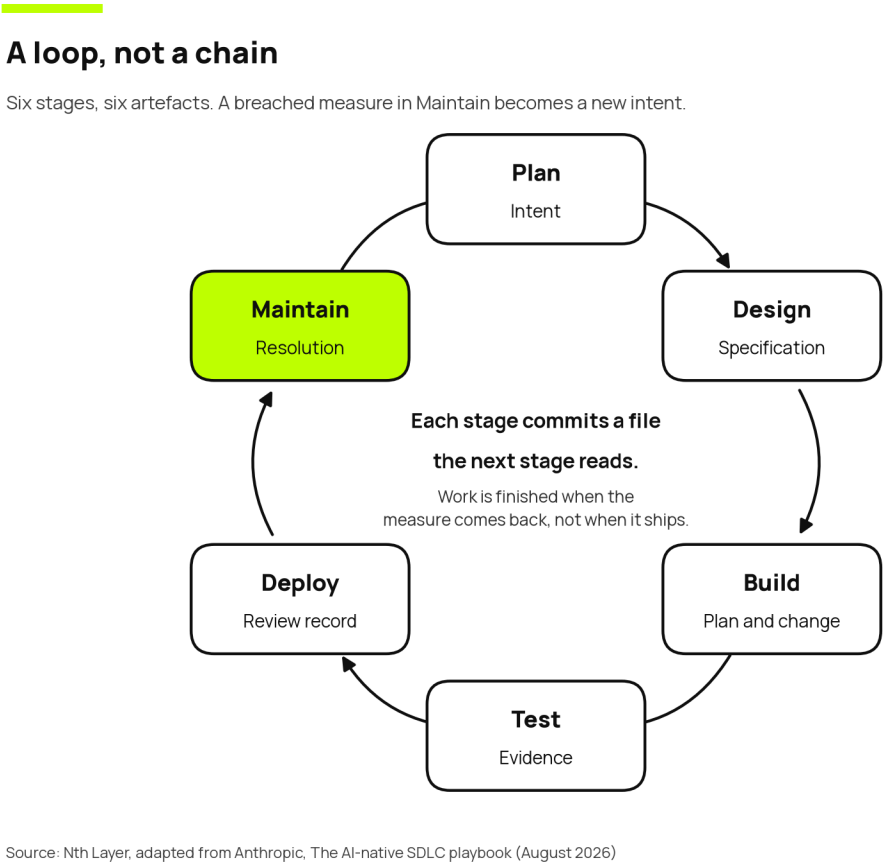


Figure 3. The six stages as a loop, each leaving an artefact.

Source: Nth Layer, adapted from Anthropic, The AI-native SDLC playbook [2].

Stage	Today	Rebuilt	Leaves behind
Plan	Written up weeks later.	The originator states a prediction with a baseline.	The hypothesis, accepted by the product owner.
Design	Policy review weeks later.	One pass, with policies applied.	The specification, reviewed by the product owner.
Build	The approach lives in one head.	Plan agreed before code; streams run in parallel.	The plan, approved by an engineer.
Test	A person checks everything at the end.	Each session verifies its work and attaches evidence.	The change and its evidence, verified by the test suite.
Deploy	Waits for a weekly board.	The same checks run on every change, tiered by risk.	The review record, approved by a named code owner.
Maintain	Waits for someone to notice.	A breached measure becomes a new hypothesis.	The verdict: won, lost, inconclusive or stopped early.

The verdict is the one artefact missing today. Work is finished when the number comes back, not when it ships. The loop and enforced controls draw on Anthropic [2] [18] and DORA [1]. Nth Layer's own contribution is the hypothesis outcomes, the 12-week programme and gates, and the levels.

Test: What Replaces Line-by-Line Review

Specification-based tests. Tests come from the agreed specification; a change passes or fails against it.

Evals or acceptance checks. Where a unit test cannot pin behaviour down, such as an AI feature, scored examples run on every change. A falling score blocks the release.

Risk-tiered review. Low-risk changes, such as copy, pass on automated checks. High-risk changes, such as payments or customer data, still get a named human reviewer.

Automated review. An AI reviewer and security scanners check every change first and attach findings as evidence. The approver judges intent and risk, not every line.

Two examples of rules that cannot be bypassed without a recorded exception: no merge to production without passing checks and a named human approver; and AI agents have no standing access to production secrets or customer data.

6. How We'll Know

Illustrative example (not a client result): a team of 12 engineers starts the pilot with changes waiting three days on average for review. The agreed threshold is to halve that wait by week 8, with defects reaching customers no worse than the baseline.

Until more cases exist, this is a model under test, and the pilot is the test. It is supported if lead time and review wait fall past the agreed threshold with defects reaching customers flat or better. It is disproved if flow measures have not moved by week 8, or defects rise.

7. Guidance Is Not a Control

Advisory controls are written guidance, such as written instructions in each repository or a wiki policy. The AI usually follows them, but nothing forces it to. Enforced controls, such as release gates and permission boundaries, block every time, for everyone.

The playbook draws the same line: "The skill makes violations rare and the hook makes them close to impossible" [2]. A skill is written guidance for the AI; a hook is automated enforcement. Anthropic's security team puts it simply: "draw the boundary around access and actions, not around a model's instructions or what we believe a model can do" [18].

- Start with two rules that genuinely cannot be bypassed without a recorded exception, such as the examples in Section 5, set centrally so they cannot be switched off on one machine.
- Keep a short register of the rules that would stop a release, each naming how it is enforced.

Accountability does not move. Approving a merge, authorising a release and accepting a risk remain human acts with names attached: "Humans remain accountable for every decision that requires judgment" [2].

8. Four Levels, Four Places to Stop

- **Level 1, tooling only.**
  AI for engineers and nothing else; Section 3 shows this does not deliver.
- **Level 2, verified delivery.**
  Every change checked the same way, enforced release gates and a measured baseline. Engineering alone.
- **Level 3, specified work.** Hypotheses and specifications agreed before any code, so policy conflicts surface on day one. It needs product and engineering to agree each specification together.
- **Level 4, closed loop.** Every hypothesis resolves, and live measures raise their own work. It needs a quarter of data and the appetite to close bets that lost.

The 12-week programme reaches Level 2 in weeks 5 to 8 and Level 3 in weeks 9 to 12. Levels 1 and 2 should show in flow measures within the 12 weeks. Levels 3 and 4, where hypotheses get verdicts, are the larger bet and need a quarter or more of data to judge.

9. Twelve Weeks, Three Gates

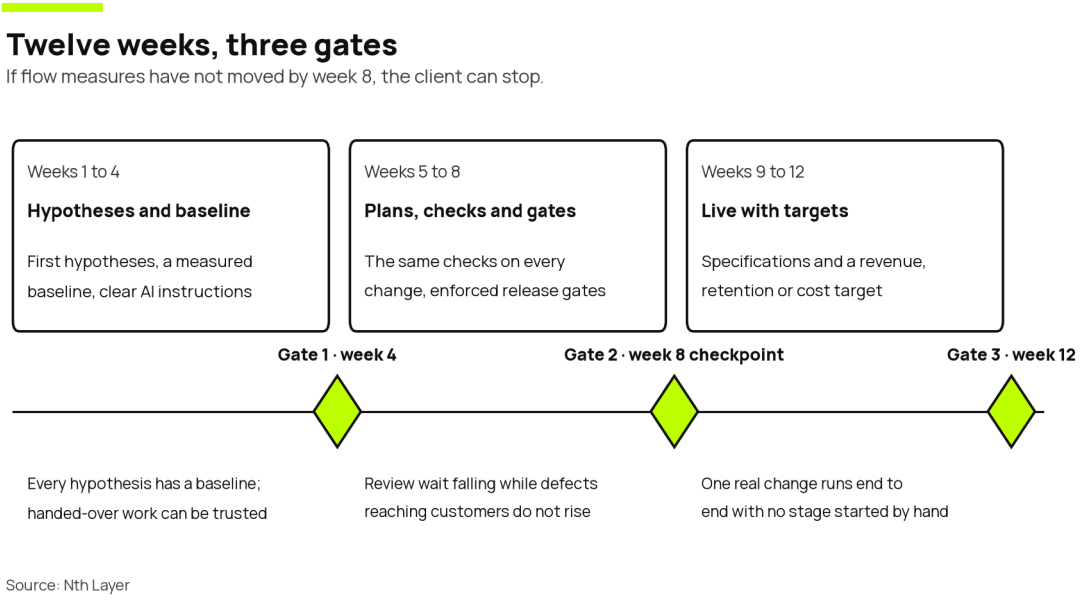


Figure 4. The 12-week programme and its three gates.

Weeks 1 to 4: hypotheses and baseline. Write the first hypotheses, measure how work moves today, and give AI written instructions in each repository, and builds and tests that run with a single command. Gate: every hypothesis has a baseline, and an engineer can trust what comes back from a handed-over task.

Weeks 5 to 8: plans, checks and gates. Plans agreed before code, the same checks on every change, enforced release gates and a named owner on every merge. Gate: review wait falls while defects reaching customers do not rise.

Weeks 9 to 12: live with targets. Policies become rules the AI applies. Gate: one real change runs end to end with no stage started by hand. Revenue and retention need a quarter or more, so are early readings.

What You Have at Week 12

- The same checks and enforced release gates run on every change.
- At least one real change has run end to end with no stage started by hand.
- Your team can run it without us.

10. Cost, Risk and People

Cost. Three named people from the existing team, an engineering lead, a platform engineer and a product owner, plus tooling costs.

Risk. Security and IP: agents have no standing access to secrets or customer data; code stays in approved tools. Compliance: SOC 2, data residency and contracts set which tools and data each stage may use. Quality: if defects reaching customers rise two months running, human review returns on everything.

People. Engineers shift time to plans and review, product owners to hypotheses. Juniors need paired work to build judgement, and involving the team in setting the rules best answers resistance.

11. Where to Start

Beyond the three questions in the summary, two more test your controls.

1. Which of our controls hold when nobody is watching? Count the ones enforced by infrastructure.
2. For our last incident, who approved the release, and does the record show what they were approving?

Then baseline these measures from systems you already have.

Measure	Source	Direction by week 12
First-pass pipeline success on AI-authored changes	Build pipeline	Rising
Time to first review	Change metadata	Minutes
Review time per change	Change metadata	Falling
Rework cycles per change	Change history	Falling
Defects reaching customers	Incident tracker	Flat or better
Hypotheses resolved, and how	Artefact history	Visible, not zero
End-to-end lead time, idea to customer	Ticketing system and release history	Falling
Deployment frequency	Release pipeline	Rising

The offer is the 12-week pilot.

12. About Nth Layer

Nth Layer works on AI-native product and engineering for B2B software companies, led by Matthew Dewstowe, who has run both sides: founder and CEO of Innovantage, sold to Bullhorn in 2012; product leader; Chief AI Officer. He works hands on with your team and tools, and leaves when they run it alone. No lock-in. Services: the AI-native pilot, product strategy and fractional or interim CPO cover. Book 30 minutes with Matthew Dewstowe at nthlayer.co.uk or matthew@nthlayer.co.uk. You leave the call with your baseline against the three questions and a clear yes or no on whether a 12-week pilot fits.

Appendix A: References

[1] Google Cloud and DORA (Nathen Harvey, Derek DeBellis), "Announcing the 2025 DORA Report: State of AI-assisted Software Development", Google Cloud blog, 23 September 2025.
https://cloud.google.com/blog/products/ai-machine-learning/announcing-the-2025-dora-report

[2] Anthropic (Louis Claxton), "The AI-native SDLC playbook", Claude blog, 21 August 2026.
https://claude.com/blog/the-ai-native-sdlc-playbook

[3] Bain & Company, "From Pilots to Payoff: Generative AI in Software Development", Technology Report 2025, 23 September 2025.
https://www.bain.com/insights/from-pilots-to-payoff-generative-ai-in-software-development-technology-report-2025/

[4] The Nth Layer, "AI-Native Product and Engineering for SaaS", nthlayer.co.uk, accessed 19 October 2026. https://nthlayer.co.uk/

[5] Sida Peng, Eirini Kalliamvakou, Peter Cihon, Mert Demirer, "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot", arXiv 2302.06590, 13 February 2023. https://arxiv.org/abs/2302.06590

[6] Anthropic (Saffron Huang and others), "How AI is transforming work at Anthropic", 2 December 2025.
https://www.anthropic.com/research/how-ai-is-transforming-work-at-anthropic

[7] Nathan Wilmers, Regina Catipon, Maxim Massenkoff, Peter McCrory, "How employment is changing at Anthropic", The Anthropic Institute technical report, 16 October 2026. https://www.anthropic.com/research/employment-at-anthropic

[8] Faros AI, "The AI Productivity Paradox Report 2025", 23 July 2025. https://www.faros.ai/blog/ai-software-engineering

[9] Atlassian (Andrew Boyagi), "Atlassian research: AI adoption is rising, but friction persists" (State of Developer Experience 2025), 9 July 2025. https://www.atlassian.com/blog/developer/developer-experience-report-2025

[10] Google Cloud and DORA, "Impact of Generative AI in Software Development" (findings from the 2024 DORA research), 2024.
https://dora.dev/ai/gen-ai-report/dora-impact-of-generative-ai-in-software-development.pdf

[11] METR (Joel Becker, Nate Rush, Beth Barnes, David Rein), "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity", 10 July 2025. https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/

[12] METR (Joel Becker, Nate Rush, Tom Cunningham, David Rein, Khalid Mahamud), "We are Changing our Developer Productivity Experiment Design", 24 February 2026. https://metr.org/blog/2026-02-24-uplift-update/

[13] Stack Overflow, "2025 Developer Survey: AI", July 2025. https://survey.stackoverflow.co/2025/ai

[14] Veracode, "2025 GenAI Code Security Report", July 2025.
https://www.veracode.com/wp-content/uploads/2025_GenAI_Code_Security_Report_Final.pdf

[15] Alan Shimel, "The AI Agent Race: At the Top of the Stretch", DevOps.com report, August 2026 (citing Futurum Group 2H 2026 Software Lifecycle Engineering survey, n=839). https://devops.com/the-ai-agent-race-is-on-but-are-we-watching-the-right-race/

[16] The Futurum Group, "Software Lifecycle Engineering Market to Reach $226 Billion by 2030 as Enterprise Hands AI the Keyboard Before the Guardrails", press release, 13 July 2026.
https://futurumgroup.com/press-release/software-lifecycle-engineering-market-to-reach-226-billion-by-2030/

[17] Zoe Hitzig, Maxim Massenkoff, Eva Lyubich, Shaoyi Zhang, Ryan Heller, Peter McCrory, "Agentic coding and persistent returns to expertise", Anthropic, 16 June 2026. https://www.anthropic.com/research/claude-code-expertise

[18] Jason Clinton, "How Anthropic secures its AI-native software development lifecycle", Claude blog, Anthropic, 21 July 2026.
https://claude.com/blog/how-anthropic-secures-its-ai-native-software-development-lifecycle

All sources accessed and checked on 18 October 2026, and nthlayer.co.uk on 19 October 2026. Charts marked "redrawn by Nth Layer" are new renderings of figures stated in the cited source and are not reproductions of the original artwork.